fMRI-DP — Remaining Issues

Reviewed `github.com/Novelex/fMRI-DP` against the paper. Ordered by severity.

BLOCKER 1 — `dyn_weight` batches wrong, silently

`Dataset.py:83` attaches `dyn_weight` after `Data(...)` construction, so PyG gives it no `__cat_dim__` and **concatenates** along dim 0 instead of stacking.

For batch_size 32, T=3:

- expected `[32, 3, 90, 90]`
- actual `[96, 90, 90]`

`GraSTI.py:73-74` then does:

```python
num_windows = dyn_weight.shape[0] // batch_size   # 96 // 32 = 3   ✓ correct
dyn_weight_reshaped = dyn_weight.reshape(batch_size, num_windows, 90, 90)
```

This happens to recover the right shape **only because PyG concatenates graph-by-graph contiguously and every subject has the same T.** It is not guaranteed by any contract — it is arithmetic luck. Add to `Dataset.py`:

```python
class WindowedData(Data):
    def __cat_dim__(self, key, value, *args, **kwargs):
        if key == 'dyn_weight':
            return None                       # stack -> [B, T, 90, 90]
        return super().__cat_dim__(key, value, *args, **kwargs)
```

Verify: load a batch of 4 known subjects, assert `dyn_weight[i]` equals subject *i*'s on-disk `correlation_matrices`. Do this before any training run.

BLOCKER 2 — signed PCC breaks Eq. 12's premise

`Dataset.py:74` loads `cropped_matrix` raw and passes it straight through as `edge_weight`. Your matrices are **signed** (correct for the archive), so ToyNet's VIB receives values in [-1, 1].

`GInfoMinMax.forward` correctly keeps this separate from the GCN weight — that fix is right and important. But Eq. 12 is:

```
A = sigmoid( (1/tau) * ( log(eps/(1-eps)) + log(W_ij/(1-W_ij)) ) )
```

which requires $W \in (0,1)$. Negative W makes `log(W/(1-W))` undefined.

Your code sidesteps this by substituting ToyNet's learned `edge_logits` for the `log(W/(1-W))` term — so nothing crashes. But then the raw PCC never enters Eq. 12 at all, and the connection to the paper's derivation is looser than the equation implies.

Decide and document one of:

- **(a)** Convert at load: `edge_weight = clip(abs(pcc), 1e-6, 1-1e-6)`. Faithful to Eq. 12, discards anticorrelation sign. State that in the methods.
- **(b)** Keep signed, and state explicitly that ToyNet's logits replace Eq. 12's `log(W/(1-W))` term as a practical implementation.

Either is defensible. Silently doing (b) while citing Eq. 12 is not.

BLOCKER 3 — the gate has no gradient path to the view learner

`GraSTIACL.py:175-184`:

```python
x, _, edge_logits = model(...)                # <- from MODEL's ToyNet
_, mu, std, edge_prod = view_learner(...)     # <- view learner's edge_logits DIS
...
gate_inputs = (gate_inputs + edge_logits) / temperature
batch_aug_edge_weight = torch.sigmoid(gate_inputs)
```

`view_learner(...)` returns `edge_logits` first, but it is thrown into `_`. The gate is built from `model`'s ToyNet instead.

You **partially** mitigated this via the shared backbone (Issue #16) — `model.net is view_learner.net`, so gradients do reach the shared ToyNet. But:

- `view_learner.mlp_edge_model` receives **zero gradient**, always. It is built in `__init__` and only used in the `dyn_weight is None` branch, which never fires.
- Eq. 15 is `min_Phi max_Theta`, with `Phi` = adjacency-encoding params and `Theta` = node-encoding params. Sharing all of them means there is no real adversarial separation — both phases optimize the same weights in opposite directions.

Fix: use the view learner's own return value:

```python
edge_logits_vl, mu, std, edge_prod = view_learner(...)
gate_inputs = (gate_inputs + edge_logits_vl) / temperature
```

Verify: print per-module grad norms per phase. `view_learner.mlp_edge_model` must be non-zero in the min phase.

BUG 4 — Beta distribution is inverted vs. Eq. 18

`TA_encoder.py:295-298`:

```python
gamma = edge_weight.mean().clamp(eps, 1 - eps)
lambda_ = torch.distributions.Beta(gamma, 1 - gamma).sample()
```

`gamma` here is **retention** (mean of A, near 1 when edges are kept), giving `E[lambda] = gamma`. So when the graph is dense, lambda is large and attention dominates.

But §3.3 says the opposite: *"as the connectivity of the graph diminishes during training, this encoder can appropriately reduce the influence of topological information and focus more on global information."* Less connectivity should mean **more** attention weight, i.e. `E[lambda] = 1 - gamma`.

Eq. 18 as written gives `E[lambda] = gamma`; the prose gives `1 - gamma`. **The paper is internally inconsistent.** The released code computes the *drop* probability and uses `Beta(1-gamma, gamma)`, i.e. it follows the prose.

Yours follows Eq. 18 and contradicts the prose. That is a legitimate choice — but pick it deliberately, document it, and **ablate both**. This is a free extra result.

Second problem, same lines: for the **unweighted view** (`edge_weight is None`), gamma is hardcoded to `1 - eps`, so `Beta(0.9999, 0.0001)` puts lambda ~ 1 almost surely. The original and augmented views therefore get systematically different mixing, which confounds every contrastive comparison. Use the same gamma for both, or state why not.

BUG 5 — `beta = np.random.beta(fin_reg, 1 - fin_reg)` can raise

`GraSTIACL.py:275`. `fin_reg` is the mean edge-drop probability. If it ever reaches exactly 0 or 1, `np.random.beta` raises `ValueError: a <= 0`. Unlikely but not impossible late in training when the gate saturates. Clamp it:

```python
fin_reg_c = float(np.clip(fin_reg, 1e-4, 1 - 1e-4))
beta = np.random.beta(fin_reg_c, 1 - fin_reg_c)
```

Also note `beta` is threaded into `TAEncoder.__init__` and `forward`, but the fusion now uses `lambda_` from Eq. 18 instead. Confirm `beta` is genuinely unused in the encoder and remove it, or you will confuse yourself later about which parameter controls the mixing.

Smaller items

- `np.nan_to_num` at `Dataset.py:61,65` — you verified the data is clean and decided to leave it. Fine. But now that the loader has been rewritten several times, an `assert`

`np.isfinite(x).all()` costs nothing and catches a future self-inflicted parsing bug. Your call.

- **`edge_logits = torch.cat(...)` in a 90-iteration loop** (`GraSTI.py:83`) — O(n^2) reallocation, 90 x batch_size times per forward. Collect into a list and `torch.cat` once. Pure speed, no correctness impact.
- **`aug_edge_weight_all_*` accumulate on CPU** inside the batch loop (`GraSTIACL.py:251`). Forces a GPU->CPU sync every batch. Accumulate on device, move once per epoch.
- **`torch.rand(edge_logits.size())` then `.to(device)`** (`:181`) — allocates on CPU then copies. Use `torch.rand(..., device=device)`.

What is already correct — do not touch

- ToyNet: `FC_mean/FC_var` map `hidden -> latent` directly. The 1x1/90x800 crash is gone.
- `get_mu_std_logits` concatenates static row (90) + dynamic rows (270) = 360 per ROI. Dynamic weights genuinely enter the forward pass and receive gradient.
- `GInfoMinMax.forward` keeps `gcn_edge_weight` and `pcc_weight` separate. This is subtle and correct — the GCN must never receive raw signed PCC, or degree normalization breaks.
- `edge_index` built via meshgrid = all 8100 pairs. Immune to the exact-zero bug that breaks the original at `GraSTI.py:75`.
- Subjects matched by filename, not `os.listdir()` position.
- Labels: ASD = 1, NC = 0. Metrics now describe patients.
- Attention is block-diagonal per subject; softmax(QK^T/sqrt(d)) matches Graphormer Eq. 4.
- `drop_last=False` with `batch.num_graphs` throughout.
- Evaluation: StratifiedKFold seeded, scaler inside GridSearchCV, AUC from `decision_function`, all metrics from `best_val_epoch`.

Order to fix

```
1. Blocker 1  __cat_dim__            -> verify batch shapes on 4 known
subjects
2. Blocker 2  decide signed vs abs   -> document the choice
3. Blocker 3  view learner's logits  -> verify mlp_edge_model grad != 0
4. Bug 4      Beta ordering          -> pick, document, ablate both
5. Bug 5      clamp fin_reg
6. Gradient gates (all six) before any real run
```

Gradient gates:

- gradient reaches `dyn_weight`

- shuffling `dyn_weight` changes the loss
- swapping dynamic for static changes the output
- `view_learner.mlp_edge_model` grad != 0 in the min phase
- gate logits originate in the view learner
- same seed twice -> identical numbers